Testes Automatizados

API de Gerenciamento de Pedidos

Tópicos Avançados em Sistemas de Informação II

Integrantes

- Pablo Neris Santiago – RF01 – Cadastro de pedido (POST /pedidos)
- Daniel Bandeira Baldini – RF02 e RF03 – Listagem e consulta (GET /pedidos e GET /pedidos/:id)
- Danillo Fernandes Gomes – RF04 – Atualização de status (PATCH /pedidos/:id/status)

Ferramentas

Jest (executor de testes) e Supertest (requisições HTTP à aplicação Express sem subir o servidor). Os testes importam o app de src/app.js e limpam o armazenamento em memória antes de cada caso (service.resetar()), para que um teste não interfira no outro.

Regras validadas

Campo	Valores aceitos
cliente	Texto não vazio, mesmo após remover espaços das extremidades
valor	Número maior que zero
tipo	entrega, retirada
prioridade	baixa, normal, alta
status (transições)	recebido -> em_preparacao | cancelado; em_preparacao -> enviado | cancelado; enviado -> entregue; entregue e cancelado são finais

Como executar

```
npm install
npm test
```

Pablo Neris Santiago

RF01 – Cadastro de pedido (POST /pedidos)
Arquivo: tests/pablo.criacao.test.js

Casos de teste

#	Cenário	Entrada	Resultado esperado
1	Criar pedido válido	cliente "Carlos Silva", tipo "entrega", valor 350, prioridade "normal"	201 – pedido com id 1 e status "recebido"
2	Cliente só com espaços	cliente " " (demais campos válidos)	400 – JSON com campo "erro"
3	Valor inválido	valor 0, depois -10, depois "350" (texto)	400 em todos – mensagem citando o valor

Código

```
// Testes feitos por: Pablo Neris Santiago
// Foco: RF01 - Cadastro de pedido (POST /pedidos)
const request = require('supertest');
const app = require('../src/app');
const service = require('../src/services/pedidoService');

beforeEach(() => service.resetar());

describe('Pablo Neris Santiago - POST /pedidos', () => {
  test('cria pedido válido e retorna 201 com status "recebido"', async () => {
    const res = await request(app).post('/pedidos').send({
      cliente: 'Carlos Silva', tipo: 'entrega', valor: 350, prioridade: 'normal',
    });

    expect(res.status).toBe(201);
    expect(res.body).toEqual({
      id: 1, cliente: 'Carlos Silva', tipo: 'entrega', valor: 350, prioridade: 'normal',
          status: 'recebido',
    });
  });

  test('retorna 400 quando o cliente contém apenas espaços', async () => {
    const res = await request(app).post('/pedidos').send({
      cliente: '    ', tipo: 'entrega', valor: 350, prioridade: 'normal',
    });

    expect(res.status).toBe(400);
    expect(res.body).toHaveProperty('erro');
  });

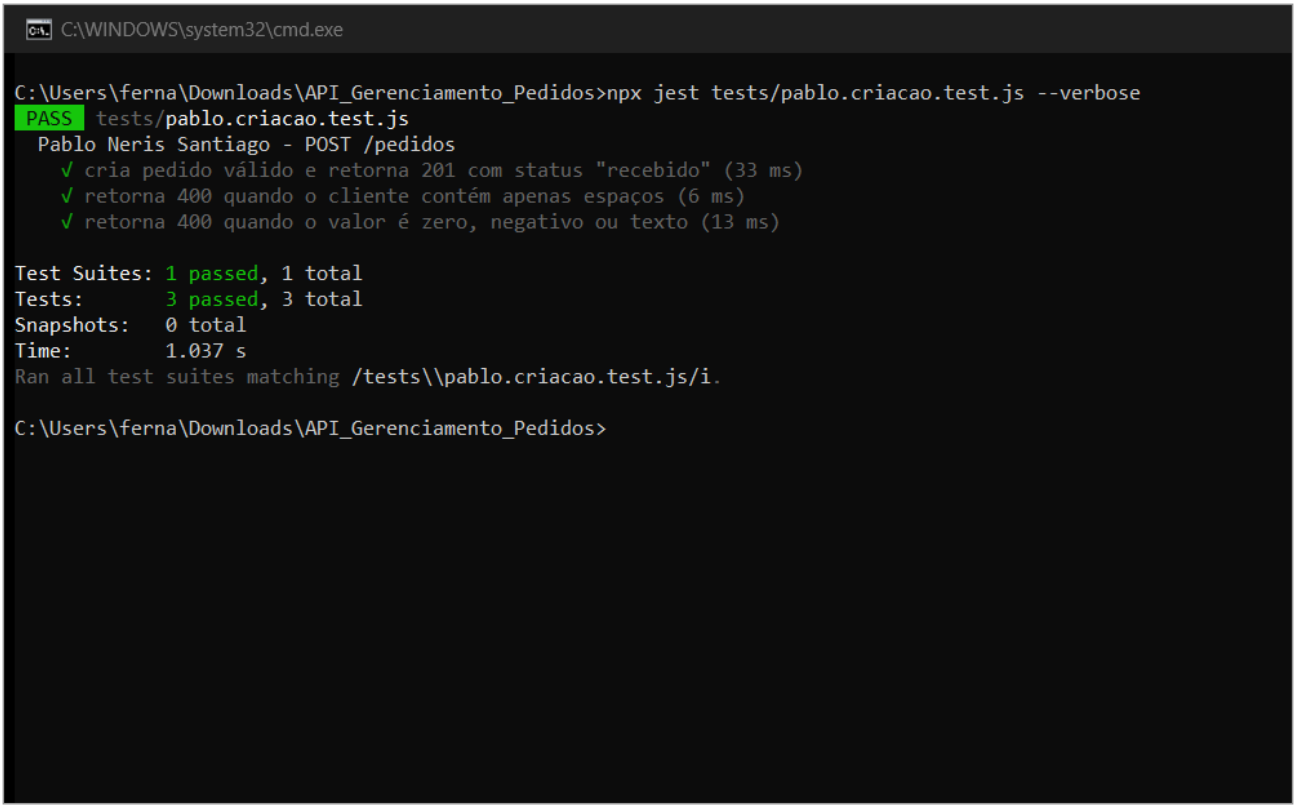
  test('retorna 400 quando o valor é zero, negativo ou texto', async () => {
    for (const valor of [0, -10, '350']) {
      const res = await request(app).post('/pedidos').send({
        cliente: 'Carlos Silva', tipo: 'entrega', valor, prioridade: 'normal',
      });

      expect(res.status).toBe(400);
      expect(res.body.erro).toMatch(/valor/i);
    }
  });
});
```

Resultado da execução

3 de 3 testes passaram.

Execução no Prompt de Comando (cmd)



Print: evidencias/testes/01-pablo-neris-santiago.png

Daniel Bandeira Baldini

RF02 e RF03 – Listagem e consulta (GET /pedidos e GET /pedidos/:id)
Arquivo: tests/daniel.consulta.test.js

Casos de teste

#	Cenário	Entrada	Resultado esperado
1	Listar pedidos	Cadastra 2 pedidos e faz GET /pedidos	200 – array com 2 pedidos na ordem de cadastro
2	Consultar ID existente	GET /pedidos/{id do pedido criado}	200 – mesmo pedido retornado na criação
3	Consultar ID inexistente	GET /pedidos/9999	404 – JSON com campo "erro"

Código

```
// Testes feitos por: Daniel Bandeira Baldini
// Foco: RF02 e RF03 - Listagem e consulta (GET /pedidos e GET /pedidos/:id)
const request = require('supertest');
const app = require('../src/app');
const service = require('../src/services/pedidoService');

const pedidoValido = { cliente: 'Ana Souza', tipo: 'retirada', valor: 120.5, prioridade:
    'alta' };

beforeEach(() => service.resetar());

describe('Daniel Bandeira Baldini - GET /pedidos', () => {
  test('lista os pedidos cadastrados e retorna 200', async () => {
    await request(app).post('/pedidos').send(pedidoValido);
    await request(app).post('/pedidos').send({ ...pedidoValido, cliente: 'Bruno Lima' });

    const res = await request(app).get('/pedidos');

    expect(res.status).toBe(200);
    expect(Array.isArray(res.body)).toBe(true);
    expect(res.body).toHaveLength(2);
    expect(res.body.map((p) => p.cliente)).toEqual(['Ana Souza', 'Bruno Lima']);
  });

  test('consulta pedido existente pelo id e retorna 200', async () => {
    const criado = await request(app).post('/pedidos').send(pedidoValido);

    const res = await request(app).get(`/pedidos/${criado.body.id}`);

    expect(res.status).toBe(200);
    expect(res.body).toEqual(criado.body);
  });

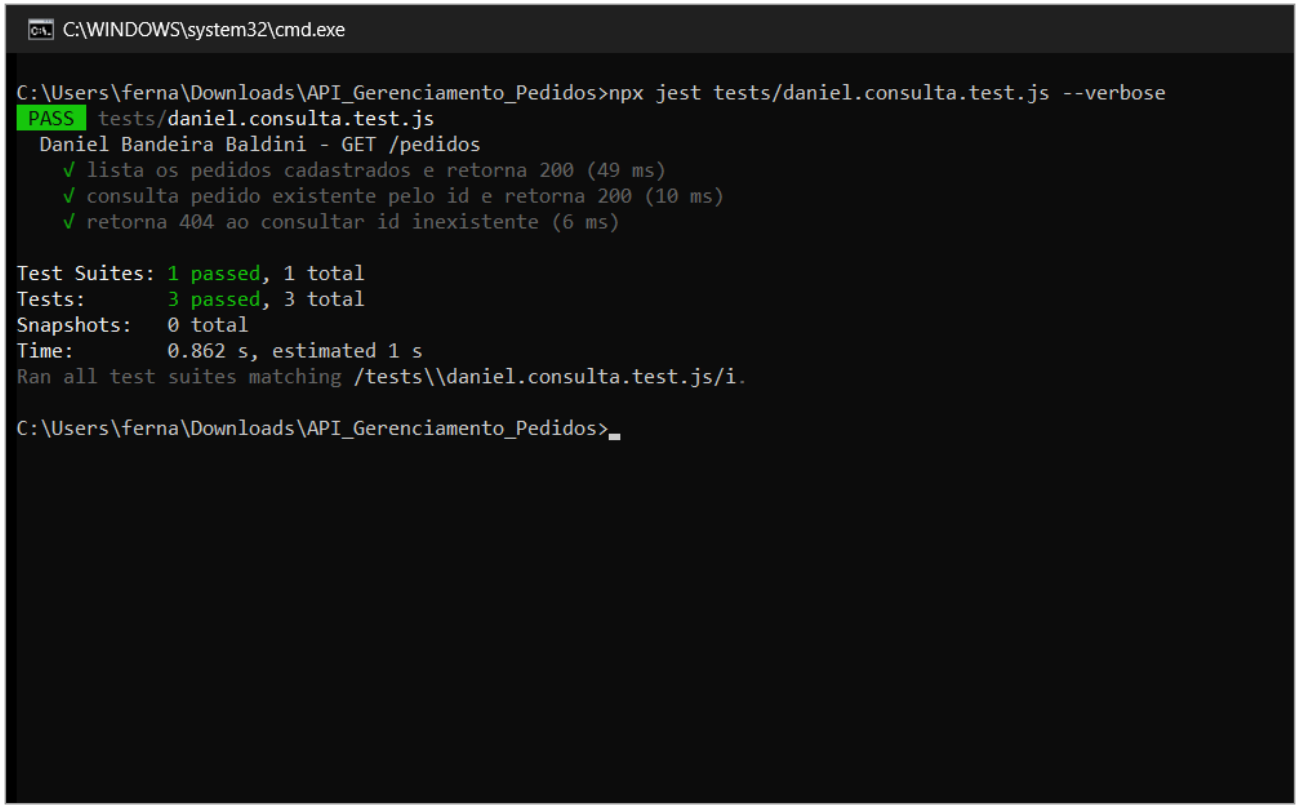
  test('retorna 404 ao consultar id inexistente', async () => {
    const res = await request(app).get('/pedidos/9999');

    expect(res.status).toBe(404);
    expect(res.body).toHaveProperty('erro');
  });
});
```

Resultado da execução

3 de 3 testes passaram.

Execução no Prompt de Comando (cmd)



Print: evidencias/testes/02-daniel-bandeira-baldini.png

Danillo Fernandes Gomes

RF04 – Atualização de status (PATCH /pedidos/:id/status)
Arquivo: tests/danillo.status.test.js

Casos de teste

#	Cenário	Entrada	Resultado esperado
1	Transição válida	{"status": "em_preparacao"} em pedido "recebido"	200 – status "em_preparacao"
2	Status/transição inválidos	{"status": "voando"} e depois {"status": "entregue"} direto de "recebido"	400 nos dois – mensagem de transição não permitida
3	ID inexistente	PATCH /pedidos/9999/status com {"status": "cancelado"}	404 – JSON com campo "erro"

Código

```
// Testes feitos por: Danillo Fernandes Gomes
// Foco: RF04 - Atualização de status (PATCH /pedidos/:id/status)
const request = require('supertest');
const app = require('../src/app');
const service = require('../src/services/pedidoService');

let pedidoId;

beforeEach(async () => {
  service.resetar();
  const res = await request(app).post('/pedidos').send({
    cliente: 'Carlos Silva', tipo: 'entrega', valor: 350, prioridade: 'normal',
  });
  pedidoId = res.body.id;
});

describe('Danillo Fernandes Gomes - PATCH /pedidos/:id/status', () => {
  test('atualiza de "recebido" para "em_preparacao" e retorna 200', async () => {
    const res = await request(app).patch(`/pedidos/${pedidoId}/status`).send({ status:
        'em_preparacao' });

    expect(res.status).toBe(200);
    expect(res.body.status).toBe('em_preparacao');
  });

  test('retorna 400 para status desconhecido ou transição não permitida', async () => {
    const desconhecido = await request(app).patch(`/pedidos/${pedidoId}/status`).send({
        status: 'voando' });
    expect(desconhecido.status).toBe(400);

    // "recebido" não pode ir direto para "entregue"
    const pulandoEtapa = await request(app).patch(`/pedidos/${pedidoId}/status`).send({
        status: 'entregue' });
    expect(pulandoEtapa.status).toBe(400);
    expect(pulandoEtapa.body.erro).toMatch(/transição/i);
  });

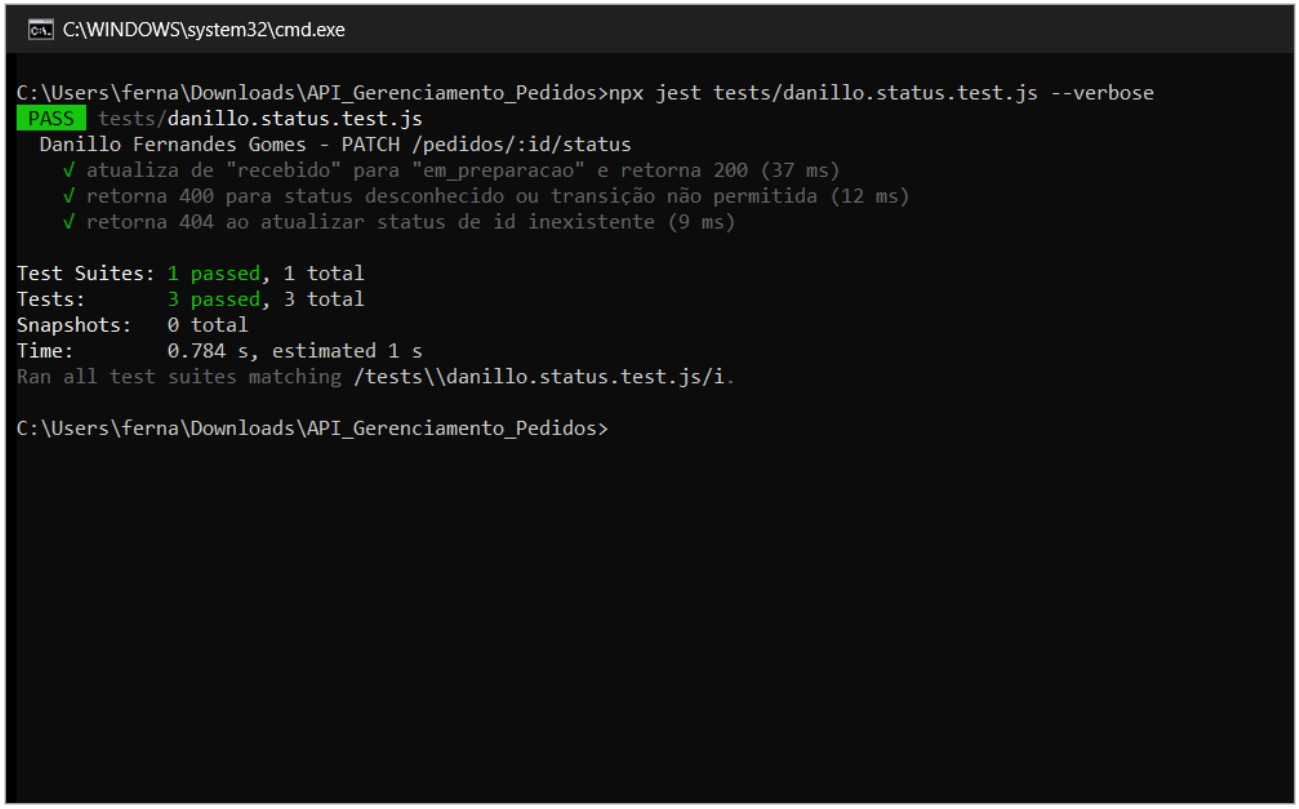
  test('retorna 404 ao atualizar status de id inexistente', async () => {
    const res = await request(app).patch('/pedidos/9999/status').send({ status:
        'cancelado' });

    expect(res.status).toBe(404);
    expect(res.body).toHaveProperty('erro');
  });
});
```

Resultado da execução

3 de 3 testes passaram.

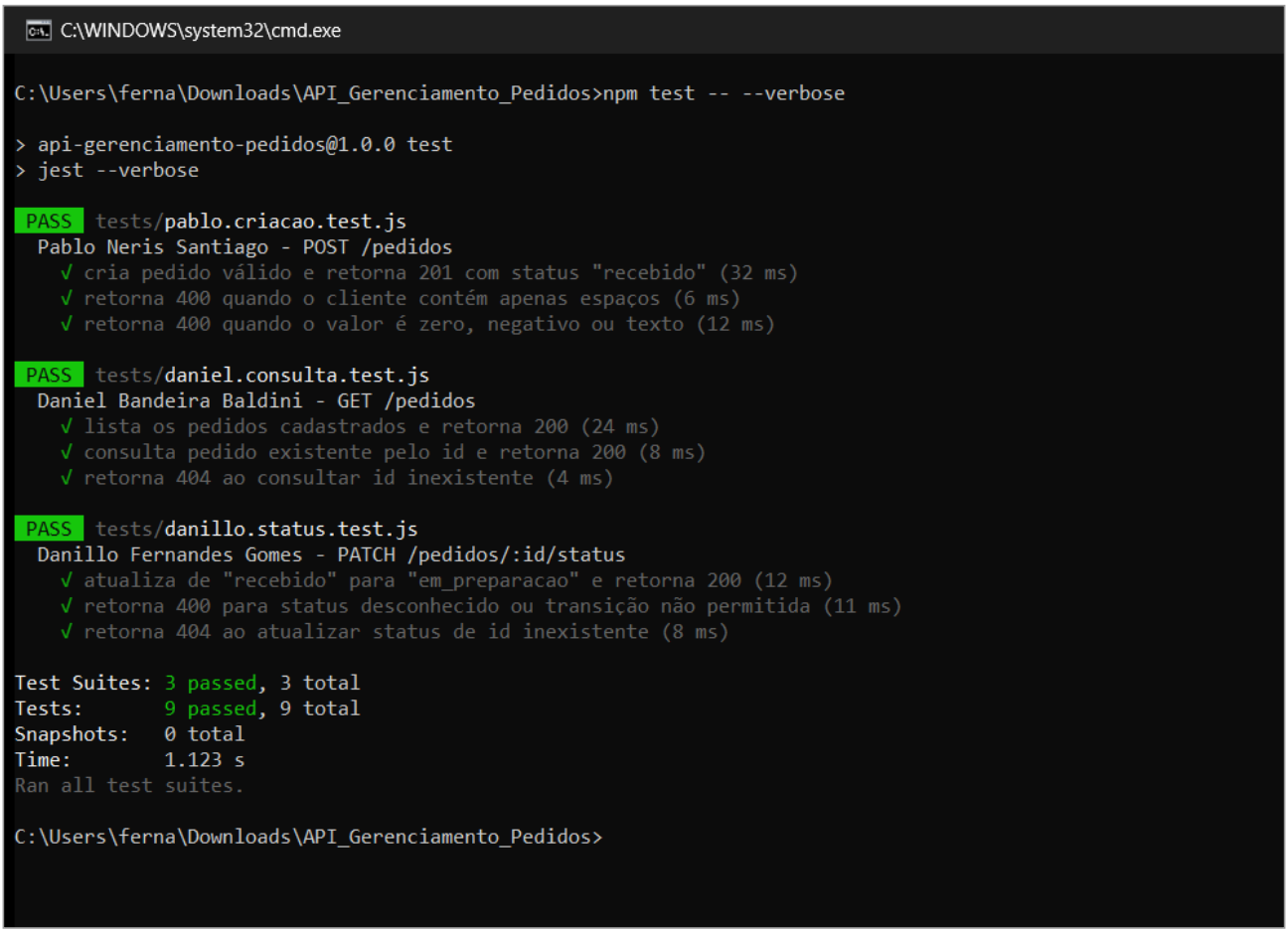
Execução no Prompt de Comando (cmd)



Print: evidencias/testes/03-danillo-fernandes-gomes.png

Resultado geral

Execução de todos os testes com npm test no Prompt de Comando (cmd):



Print: evidencias/testes/04-todos-os-testes.png

Integrante	Endpoint(s)	Testes	Situação
Pablo Neris Santiago	POST /pedidos	3	Aprovados
Daniel Bandeira Baldini	GET /pedidos e GET /pedidos/:id	3	Aprovados
Danillo Fernandes Gomes	PATCH /pedidos/:id/status	3	Aprovados